Technische ontwerp

Inhoudsopgave

---

1. Inleiding
2. Systeemeisen
3. Architectuur & Componenten
4. Data Model
5. Technologieën & Frameworks
6. Beveiliging
7. Functionaliteiten & Logica
8. Gebruikersinterfaces (UI)
9. Testplan

Inleiding

De website voor Het Utrechtse Archief wordt gemaakt voor een tentoonstelling waar een oude video met interactie wordt laten zien. In het TO gaan we het hebben over met welke technieken we gaan werken, data structuur, frameworks, wireframes en andere technische dingen die nodig zijn voor het bouwen van de website.

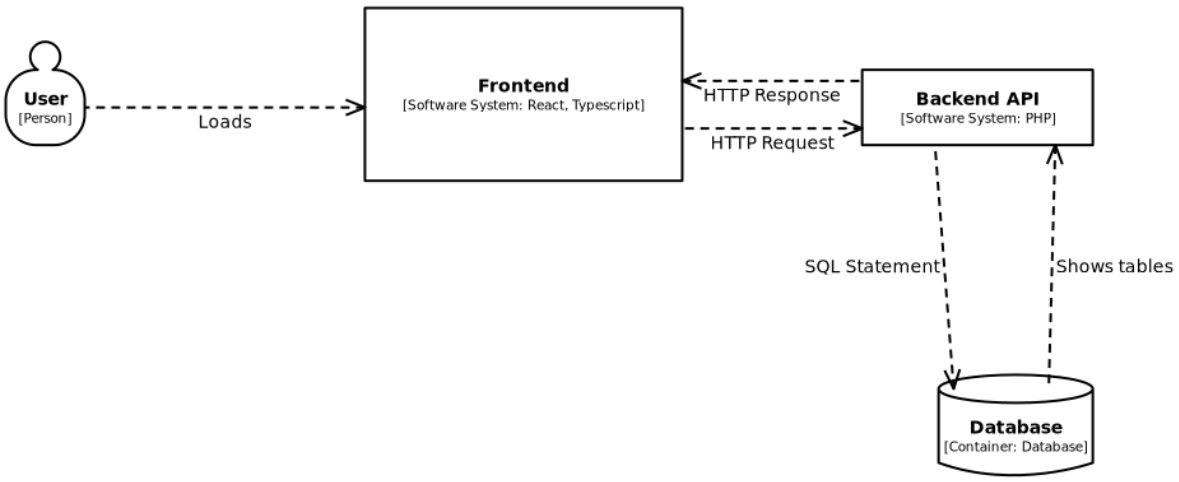
Systeemeisen

De eisen voor software zijn als volgend, de server draait op PHP en SQL. Dat betekent dat er geen Javascript via Node of Bun kan draaien. De frontend kan wel draaien via Javascript.

De browser die wij gaan volgen is GoogleChrome omdat die de laatste features heeft van Javascript en CSS

Architectuur & Componenten

Systeem diagram



Frontend

De Frontend maakt gebruik van React & Typescript. Dit doen wij omdat wij met Typescript een predictable environment hebben en daardoor voorkomen dat er bugs komen die wij niet verwachten. React gebruiken wij omdat het fijn werkt door te integratie van Javascript/Typescript.

Backend

Voor Backend gebruiken wij PHP waarom doen we dat? Omdat wij een simpele API nodig hebben. Omdat wij niet zo veel data nodig hebben is dit heel simpel en efficient.

Database

Voor Database gaan wij MySQL gebruiken. We gebruiken dit omdat het heel erg supported is samen met PHP. Deze twee talen gaan hand in hand.

Data Model



We hebben 1 tabel gemaakt met alle data. Hierin zetten we alle data die verplicht en optioneel is.
Als eerst kan je de tijd aangeven van wanneer een informatie punt zich moet afspelen, in de body kan je alle informatie over een punt inzetten, bij de ambient_path zet je eventueel een bestanden pad voor een achtergrond geluid zoals een tram die over een brug rijdt, bij de voice_path kan je een pad geven van een audio bestand van een inspreker die de informatie voorleest. Voor de img_path kan je een afbeelding toevoegen die bij de informatie kan worden gezet.

Technologieën & Frameworks

Frontend

- React typescript/Tailwind worden gebruikt voor de basisstructuur en opmaak van de website.
- Typescript word gebruikt voor interactiviteit en dynamische elementen

Backend

Voor de back end hebben we gekozen voor PHP, omdat we daar beide mee bekend zijn. Om te communiceren met de database gaan we een API maken.

CMS

We willen een eigen CMS maken waarbij je informatie punten kan wijzigen, informatie punten toevoegen, mogelijkheid om audio toe te voegen: background en narrator. Je kan er ook voor kiezen om een informatie punt te verwijderen.

Beveiliging

Inlogbeveiliging

Voor het inloggen bij het CMS gaan we de user een token laten invullen. We hebben maar 1 token, omdat we niet verschillende permissions hoeven te geven.

Inputvalidatie

Om SQL injection te voorkomen gebruiken we prepared statements, zodat je niet dingen in de database kan zien zonder een inlog gegevens te hebben. Voor XXS-aanvallen checken we de validatie van de request in de back-end.

Databasebeveiliging

De klant krijgt een token waarmee die alleen data mag aanpassen/verwijderen/toepassen. De klant kan niet hele tabellen verwijderen. Verder gebruiken we nog SQL prepared statements.

Functionaliteiten & Logica

Frontend

- Kaart
- Media Controls
- Progress tracker
- Info Points

Backend

- Login
- Api processing
- Deleting/Selecting/Editing
- File uploading

Wij gaan een CMS maken om de data aan te passen, hier kan de admin files uploaden en aanpassen of verwijderen.

Gebruikersinterfaces (UI)

Gebruikersinterfaces

We hebben op de main page is er een side-balk voor de progress van de video en points of intrest. Aan de onderkant zijn er media controls en een kaart. Dit laat zien waar de boot nu naartoe gaat. Bij de route komen er punten op de route die belangrijk zijn. Op die punten komt er informatie te zien.

Bij het CMS is het een one page view, je kan makkelijk dan een lijstje zien en daar de punten aanpassen. Dan krijg je een form om het aan te passen of te verwijderen. Ook zit er een login scherm bij.

Testplan

Wanneer een branch klaar is gaan we de functies die bij die branch zijn toegevoegd goed testen, pas als alles goed is mergen we het naar de dev branch. Wanneer we alles hebben kijken we op de dev branch goed of alles werkt: Werken de knoppen goed, is het responsive, werkt het op verschillende browsers en of het werkt op verschillende computers.